

06026240 INTELLIGENT SYSTEM DEVELOPMENT

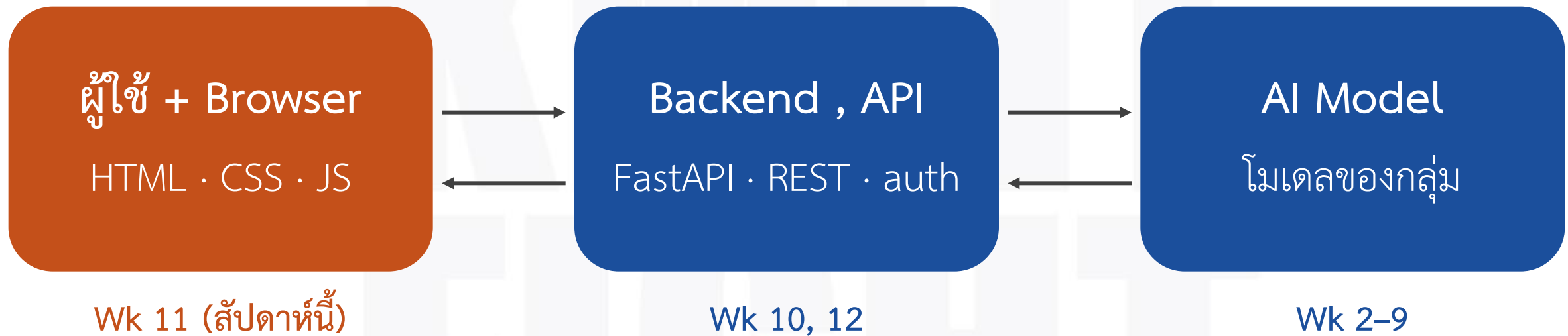
Chapter XI Front-End Development

with Dr. Bhattarabhorn Wattanacheep

Front-end development primarily focuses on user experience. Using the related coding and design techniques, you as front-end developers build the elements of an application that are directly accessed by end-users with a goal of rendering the entire interface elegant, easy to use, fast, and secure, fostering user engagement and interaction.

- เป้าหมาย:
 - สร้างประสบการณ์การใช้งานที่ดี
 - ทำให้เว็บเพจมีความสวยงามและใช้งานง่าย

Frontend คือส่วนที่ผู้ใช้เห็น ส่วนโมเดลทำงานอยู่หลัง API



- **ส่งไป:** รูป/ข้อความจากผู้ใช้ เป็น HTTP request (JSON หรือ FormData)
- **รับกลับ:** ผลทำนาย + ค่าความมั่นใจ เป็น JSON แล้ว JS นำมาแสดงบนหน้าเว็บ

ตัวอย่าง

บทบาท:

กำหนดโครงสร้างของเนื้อหาเว็บเพจ

หลักการ:

HTML เป็นภาษาที่ใช้ tag ในการจัด
ระเบียบเนื้อหา การกำหนดโครงสร้างและเนื้อหา

เช่น <header>, <footer>, <article>, <section>

เวอร์ชันปัจจุบันของ HTML คือ HTML5

html

Copy code

```
<!DOCTYPE html>
<html>
<head>
  <title>My Website</title>
</head>
<body>
  <header>
    <h1>Welcome to My Website</h1>
  </header>
  <main>
    <p>This is a paragraph.</p>
    <a href="#">Click here</a>
  </main>
  <footer>
    <p>&copy; 2024 My Website</p>
  </footer>
</body>
</html>
```

ความแตกต่างระหว่างองค์ประกอบ Inline และ Block ใน HTML

องค์ประกอบ **Inline** ใน HTML คือองค์ประกอบที่ไม่ได้เริ่มบรรทัดใหม่ทุกครั้ง และใช้พื้นที่และความกว้างเท่ากับเนื้อหา

ตัวอย่าง: `<a>`, `<strong>`, `<img>`, `<input>` เป็นต้น

องค์ประกอบ **Block** จะเริ่มต้นจากบรรทัดใหม่โดยอัตโนมัติ

ตัวอย่าง: `<div>`, `<h1>` ถึง `<h6>`, `<p>`, `<table>` เป็นต้น

การเขียน CSS และ JavaScript ใน HTML มี 3 รูปแบบ

- 1) Inline
- 2) Internal
- 3) External

แนวทางปฏิบัติที่ดี ในการใช้ JavaScript และ CSS คือ

- ให้ใช้ไฟล์ภายนอกเสมอสำหรับการกำหนด JavaScript และ CSS ที่มี นามสกุล .js และ.css ตามลำดับ
- ควรเชื่อมโยงไฟล์ CSS ไว้ภายในแท็ก <head> ของเอกสาร HTML เสมอ
- หลีกเลี่ยงการสร้างตัวแปร global และ ใช้ชื่อที่มีความหมายในการกำหนดตัวแปร

บทบาท:

การออกแบบและตกแต่งเว็บเพจ

หลักการ:

CSS ใช้ในการจัดรูปแบบและ
ปรับปรุงลักษณะของ HTML = ควบคุมสไตล์
เช่น สี, ขนาด, ระยะห่าง

ตัวอย่าง

```
css Copy code

body {
  font-family: Arial, sans-serif;
}
header {
  background-color: #f8f9fa;
  padding: 10px;
}
h1 {
  color: #333;
}
p {
  line-height: 1.6;
}
a {
  text-decoration: none;
  color: #007bff;
}
```

Responsive Design

หลักการ:

- ออกแบบให้เว็บไซต์สามารถแสดงผลได้ดีในทุกขนาดหน้าจอ

เทคนิค:

- Media Queries: ใช้ในการกำหนดรูปแบบที่แตกต่างกันสำหรับขนาดหน้าจอที่แตกต่างกัน
- Flexbox & Grid Layout: เครื่องมือสำหรับการจัดตำแหน่งและจัดระเบียบเนื้อหา

```
CSS

@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
}
```

```
CSS

/* จอใหญ่: เรียงแนวนอน */
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 16px;
}
```

CSS = Presentation Layer

- CSS ใช้กำหนดหน้าตาและการจัดวางของ HTML
- ควบคุมลักษณะ เช่น สี ขนาด ระยะห่าง ตำแหน่ง
- CSS *ไม่สามารถ* ตัดสินใจเองได้ว่า “เมื่อไหร่” จะเปลี่ยน

```
.hidden {
    display: none;
}
```

```
.active {
    background-color: #007bff;
    color: white;
}
```

JavaScript ควรควบคุม “สถานะ” ไม่ใช่ “สี”

- ❌ ไม่แนะนำให้ JS เปลี่ยน style ที่ละบรรทัด
- ✅ แนะนำให้ JS เพิ่ม / ลบ class
- ให้ CSS เป็นคนดูแลหน้าตาแทน
 - `element.classList.add('active');`

JavaScript + CSS = Interactive Web

- JavaScript ตรวจจับเหตุการณ์ (event)
- JavaScript เปลี่ยน class ของ HTML
- CSS แสดงผลตาม class ที่เปลี่ยน

```
button.addEventListener('click', () => {  
    box.classList.toggle('active');  
});
```

JavaScript Errors ที่พบบ่อย

- ✗ ReferenceError
เรียกใช้ตัวแปรที่ไม่ได้ประกาศ เช่น resultBox กับ resultbox
- ✗ TypeError
ใช้งานข้อมูลผิดประเภท
- ✗ SyntaxError
โค้ดเขียนผิดรูปแบบ
- ✗ Runtime Error
โค้ดถูกต้อง แต่ฟังก์ชันรันจริง

อ่านโค้ด .then().catch() และ try / catch ที่ละขั้น

.then().catch() ใช้กับ Promise

```
fetch(url)

// 1. ได้คำตอบ res แล้วแปลงเป็น JSON
.then(res => res.json())

// 2. ได้ข้อมูลจริงใน data แล้วแสดงผล
.then(data => console.log(data))

// 3. ถ้าขึ้นไหนพัง จะกระโดดมาที่นี่
.catch(error => console.log(error));
```

try / catch ใช้กับ async / await

```
async function loadData() {
  try {
    // 1. รอจนได้คำตอบ เก็บไว้ใน res
    let res = await fetch(url);
    // 2. รอแปลงเป็น JSON แล้วแสดงผล
    let data = await res.json();
    console.log(data);
  } catch (error) {
    // 3. บรรทัดไหนใน try พัง มาที่นี่
    console.log(error);
  }
}
```

ถ้าสำเร็จ

ทำงานจนจบ แล้วแสดงข้อมูลใน **data**

ถ้าผิดพลาด

เน็ตหลุด หรือ URL ผิด จะข้ามไปที่ **catch**
แล้วแสดง error

ระวัง: 404 / 500

fetch ถือว่าสำเร็จ ไม่เข้า **catch** ต้องเช็ค
res.ok เอง

หน้าเว็บเรียกโมเดลด้วย fetch แล้วรอ JSON กลับมา

```
const form = new FormData();
form.append("file", input.files[0]);

const res = await fetch(
  "http://localhost:8000/predict",
  { method: "POST", body: form }
);
const data = await res.json();

result.textContent =
  `${data.label} (${data.confidence}%)`;
```

3 ขั้นตอนที่ทุกกลุ่มต้องมี

1. เก็บ input ของผู้ใช้ใส่ FormData (ไฟล์) หรือ object (ข้อความ)
2. fetch ไปที่ endpoint ของ backend แล้วใช้ await รอผล
3. แปลงเป็น JSON แล้วเขียนผลลง DOM

ถ้าส่งเป็น JSON: ใส่ headers Content-Type: application/json และ JSON.stringify(body)

ตกลง API contract ก่อน แล้วสองฝั่งทำงานคู่ขนานได้

Request (frontend ส่ง)

```
POST /predict
body: FormData

file: <รูปภาพ .jpg/.png>
```

Response: สำเร็จ

```
200 OK
{
  "label": "cat",
  "confidence": 92.4,
  "model": "cnn-v2"
}
```

Response: ผิดพลาด

```
400 Bad Request
{
  "error": "ไฟล์ไม่ใช่รูปภาพ"
}
```

- ต้องตกลง 5 เรื่อง: endpoint, method, ชื่อ field, ชนิดข้อมูล (เช่น confidence เป็น 0–1 หรือ 0–100), และ รูปแบบ error
- เขียนเก็บไว้ใน README ของ repo กลุ่ม ให้ทุกคนและ AI ที่ช่วยเขียนโค้ดใช้ไฟล์เดียวกัน
- Backend ยังไม่เสร็จ? ให้ frontend ใช้ JSON ปลอมตาม contract นี้ไปก่อน

UI ของระบบ AI ต้องออกแบบให้ครบ 4 สถานะ

1 Idle	2 Loading	3 Success	4 Error
ยังไม่ได้ส่ง บอกผู้ใช้งานว่าต้องอัปโหลดอะไร ชนิดไฟล์และขนาดที่รับได้	รอโมเดล แสดง spinner/ข้อความ และปิดปุ่มกันกดซ้ำ	ได้ผลแล้ว แสดงผลทำนาย + confidence ให้อ่านเข้าใจ	พลาด บอกว่าผิดอะไร และผู้ใช้งาน ทำอะไรต่อ

- โมเดลตอบช้าได้หลายวินาที: ถ้าไม่มีสถานะ Loading ผู้ใช้จะคิดว่าเว็บค้างแล้วกดซ้ำ
- confidence ต่ำ: บอกผู้ใช้งานตรงๆ เช่น "ไม่แน่ใจ ลองรูปที่ชัดกว่านี้" แทนการแสดงคำตอบผิดอย่างมั่นใจ
- **AI มักเจอ UI มาแค่สถานะ Success:** อีก 3 สถานะต้องสั่งเพิ่มเองเสมอ

โค้ดฝั่ง frontend ทุกบรรทัด ผู้ใช้เปิดอ่านได้

1. API key ต้องอยู่ที่ backend เท่านั้น

ผิด: ใครเปิด DevTools ก็เห็น key

```
fetch("https://api.openai.com/...", {  
  headers: {  
    Authorization: "Bearer sk-..."  
  }})
```

ถูก: เรียก backend ของกลุ่ม ให้ backend ถือ key

```
fetch("/api/chat", { method: "POST", body })
```

2. ข้อความจากโมเดล อย่าใส่ด้วย innerHTML

ผิด: ถ้าโมเดลตอบเป็น `<script>` มันจะรันทันที

```
result.innerHTML = data.answer;
```

ถูก: แสดงเป็นข้อความธรรมดา

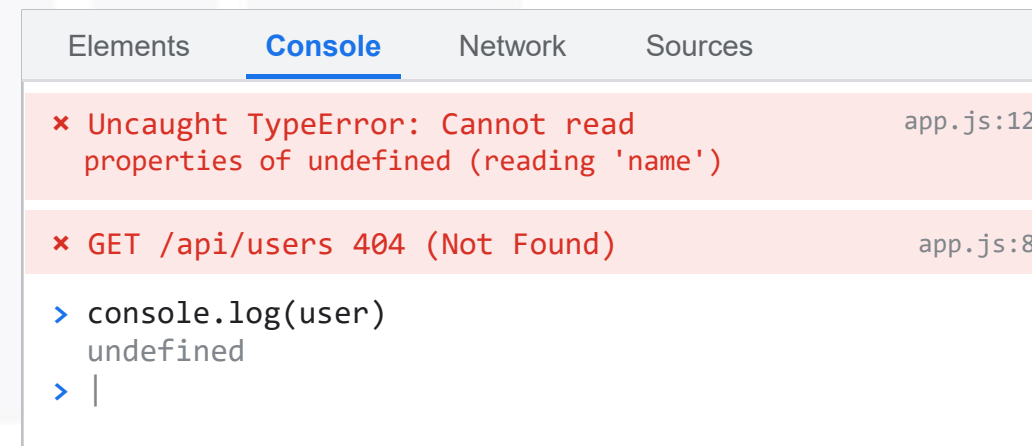
```
result.textContent = data.answer;
```

- ไฟล์ .env เก็บ key ไว้ฝั่ง backend และใส่ใน .gitignore ห้าม push ขึ้น GitHub
- ตรวจ input ฝั่ง backend จำเสมอ: การเช็คใน JS ช่วยเรื่อง UX แต่ผู้ใช้ข้ามได้
- **โค้ดจาก AI** มักใส่ key ลงในไฟล์ JS ตรงๆ เพราะเร็วที่สุด ต้องตรวจทุกครั้ง

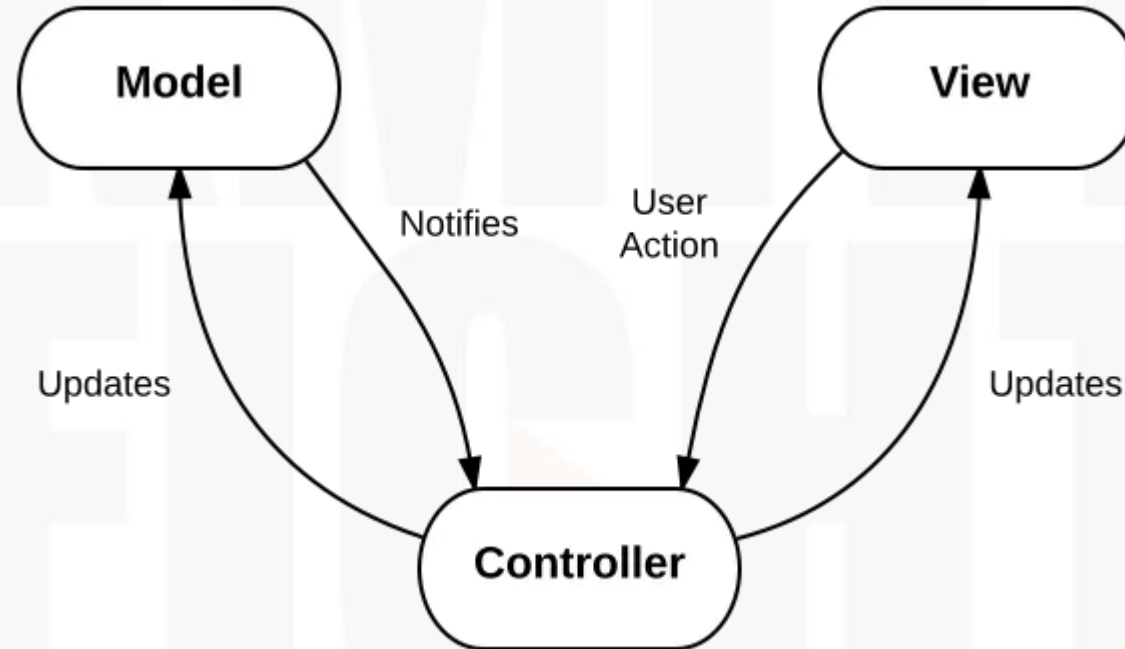
ใช้ DevTools ตรวจสอบโค้ดที่ AI เขียนทำงานจริงไหม

Elements	Console	Network
ใช้ดู ดู HTML/CSS ที่ render จริง	ใช้ดู ดู error และค่าตัวแปร	ใช้ดู ดู request/response จริง
ปัญหาที่เจอบ่อย ปุ่มหาย, layout เพี้ยน, class ไม่ติด	ปัญหาที่เจอบ่อย TypeError, undefined, CORS error	ปัญหาที่เจอบ่อย ส่งผิด URL, status 404/500, field ไม่ตรง

- เปิด DevTools: F12 (Windows) หรือ Cmd + Option + I (Mac)
- ลำดับการหาปัญหา: Console → Network (status อะไร) → Elements
- **ถาม AI ให้ได้ผล:** แขนบ error + response จาก Network + โค้ดส่วนนั้น



Model-View-Controller เป็น Software Design Pattern หรือแนวทางการออกแบบซอฟต์แวร์ หรือ Framework



<https://commons.wikimedia.org/wiki/File:MVC-basic.svg>

จุดประสงค์ของ MVC

- เพื่อแยกโค้ดออกเป็นส่วน ทำให้เราเปลี่ยนแปลงบางส่วนของโค้ดได้ โดยไม่กระทบกับส่วนอื่น
- ทำให้ Maintenance โค้ดที่หลังได้ง่าย
- ทำให้โค้ดสามารถถูกเขียนพร้อมกันโดยโปรแกรมเมอร์หลายคนได้

สรุป

- MVC เป็น Design Pattern เป็นรูปแบบการออกแบบ ไม่ได้เป็นกฎตายตัว ดังนั้นสุดท้ายแล้วจะออกแบบอย่างไรก็ขึ้นกับทีมของเรา และการตอบโจทย์ เพียงแต่ว่า MVC เป็นหนึ่งใน Design Pattern หรือ Framework การออกแบบที่นิยมใช้ในหลาย ๆ แห่ง
- MVC เองก็ถูกพัฒนาไปต่อ เป็น MVA, MVP, MVVM, MVT ฯลฯ

- Frameworks:
 - **React:** สร้าง UI แบบ components (React Foundation)
 - **Next.js:** framework ต่อยอดจาก React
 - **Angular:** framework ครบวงจร (Google)
 - **Vue.js:** เรียบง่าย เหมาะกับมือใหม่
 - **Svelte:** เขียนสั้น เร็ว ไม่ใช่ virtual DOM
- Libraries:
 - **jQuery:** จัดการ DOM และ AJAX ง่ายขึ้น
 - **Lodash:** ฟังก์ชันช่วยจัดการข้อมูล
- Build tools:
 - **Vite:** สร้างและรันโปรเจกต์ได้เร็ว

CSS Framework สำหรับ Front-end Developer

CSS Framework คือชุดคลาส CSS ที่เขียนและออกแบบไว้ล่วงหน้า ช่วยให้สร้างหน้าเว็บที่เป็นมาตรฐานและรองรับทุกขนาดหน้าจอได้รวดเร็ว โดยไม่ต้องเขียน CSS เองทั้งหมด

Tailwind CSS

Utility-first

ประกอบคลาสเล็ก ๆ
ออกแบบได้อิสระ

Bootstrap

Component-based

ส่วนประกอบสำเร็จรูป มี
JavaScript

Bulma

Component-based

CSS ล้วน เรียนรู้ง่าย

Foundation

Component-based

ปรับแต่งได้ละเอียด
สำหรับเว็บขนาดใหญ่

- หลักการ:
 - การทำให้เว็บไซต์โหลดเร็วและทำงานได้ดี
- เทคนิค:
 - **Minification:** การลดขนาดของไฟล์ JavaScript และ CSS
 - **Lazy Loading:** โหลดเนื้อหาภาพหรือข้อมูลเมื่อจำเป็น
 - **Caching:** การเก็บข้อมูลชั่วคราวเพื่อลดเวลาโหลด

Code Quality:

- การเขียนโค้ดที่สะอาดและมีการจัดระเบียบ

Version Control:

- ใช้ **Git** และ **GitHub** สำหรับการจัดการเวอร์ชันและการทำงานร่วมกัน

Testing:

- ทดสอบเว็บไซต์ในหลายเบราว์เซอร์และอุปกรณ์

- HTML = โครง, CSS = หน้าตา, JS = ควบคุมสถานะและเรียก API
- Frontend ต่อกับโมเดลผ่าน **API contract**: ส่ง request แล้วรับ JSON
- UI ของระบบ AI ต้องมี **ครบ 4 สถานะ** และห้ามมี API key
- ใช้ AI เขียนโค้ดได้ แต่ต้องตรวจด้วย **DevTools** และอธิบายได้ทุกบรรทัด

Lab: เว็บไซต์ที่ส่งข้อมูลให้โมเดลของกลุ่มและแสดงผลได้

1 Wireframe

มีหน้าอะไรบ้าง ใครเห็นหน้าไหน วาดบนกระดาษหรือ Figma ก็ได้ (up git)

2 Contract

เขียน contract ใน README (up git)

3 index.html เชื่อม UI

fetch ไปที่ API จริง และมีครบ 4 สถานะ (up git)

- ส่งท้ายคาบ: push ขึ้น repo กลุ่ม: wireframe, README, index.html, style.css, app.js (ถ้ากลุ่มไหนทำเกินแล้ว บางส่วน เลยทำให้ไม่ได้ push บางไฟล์ สามารถเข้าใจได้)

การเรียนรู้ต่อไป 5/10/2026

- เตรียมระบบให้พร้อม สำหรับกลุ่มใดต้องการทดสอบ challenge
- การทดสอบจะให้ทดสอบ ในคาบวันนั้นภายในเวลา 10.00-12.00 เท่านั้น (กลุ่มไหนจองก่อนมีสิทธิก่อน และ อาจารย์จะให้ challenge กลุ่มถัดไปได้ ก็ต่อเมื่อ กลุ่มก่อนหน้านี้ เสร็จแล้ว หากเกินเวลาจะถือว่าไม่ได้ต้องการทดสอบ)
- หากมีการติด error หรือ bug ตอนกำลังทดสอบ จะถือว่าการทดสอบเป็นโมฆะ คะแนน challenge=0 และ ยุติการทดสอบ หากกลุ่มดังกล่าวต้องการทดสอบใหม่ จะต้องไปต่อท้ายแถวของการจองทำ challenge
- สิ่งที่ต้องส่งให้.เพื่อไปวัดประสิทธิภาพ คือ
 - ไฟล์ผล json ที่ได้จาก transcript (ด้วย Accuracy, CER, WER)
 - ผลคำตอบ ที่ได้จาก การถาม curriculum book (หาก eval เองก็ส่งผล eval ตัวเองมาเพิ่มเติมได้ ซึ่งมีผลต่อการตัดสิน ของอ. เนื่องจากของอ.จะใช้ LLM judge)]
 - ต้องส่งลงใน discord กลุ่มทันทีหลังทดสอบเสร็จ เพื่อให้.ไปมอบ challenge ให้กลุ่มอื่นต่อ